

Output:

Dogs

GET

/api/Dog

POST

/api/Dog

GET

/api/Dog/{id}

PUT

/api/Dog/{id}

DELETE

/api/Dog/{id}

Dogs

GET

/api/Dog

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5276/api/Dog' \
-H 'accept: text/plain'
```

Request URL

```
http://localhost:5276/api/Dog
```

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "name": "Golden Retriever", "price": 15000 }, { "id": 2, "name": "German Shepard", "price": 10000 }, { "id": 3, "name": "Indian Dog", "price": 100 }]</pre> Download

Program.cs:

```
using sampleDog;
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOpenApi();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddControllers();

string connectionString = "Data Source = PTPLL487;Initial Catalog = Training;Integrated Security = true;TrustServerCertificate = True;";
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "Pet API V1");
    });
}

app.UseHttpsRedirection();

app.MapControllers();
app.Run();
```

Context:

```
namespace sampleDog;
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options
):base(options) {}

    public DbSet<Dog> Dogs { get; set; }
}
```

Controller:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using sampleDog;

namespace sampleDog.Controllers
{
    [Route("api/Dog")]
    [ApiController]
    public class DogsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public DogsController(AppDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Dog>>> GetDogs()
        {
            return await _context.Dogs.ToListAsync();
        }
    }
}
```

```
[HttpGet("{id}")]
public async Task<ActionResult<Dog>> GetDog(int id)
{
    var dog = await _context.Dogs.FindAsync(id);

    if (dog == null)
    {
        return NotFound();
    }

    return dog;
}

[HttpPost]
public async Task<ActionResult<Dog>> PostDog(Dog dog)
{
    _context.Dogs.Add(dog);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetDog), new { id = dog.ID },
dog);
}

[HttpPut("{id}")]
public async Task<IActionResult> PutDog(int id, Dog dog)
{
    if (id != dog.ID)
    {
        return BadRequest();
    }

    _context.Entry(dog).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Dogs.Any(e => e.ID == id))

```

```
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }

    return NoContent();
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteDog(int id)
{
    var dog = await _context.Dogs.FindAsync(id);
    if (dog == null)
    {
        return NotFound();
    }

    _context.Dogs.Remove(dog);
    await _context.SaveChangesAsync();

    return NoContent();
}
}
```

Model:

```
public class Dog
{
    public int ID { get; set; }
    public string Name { get; set; }

    public int Price { get; set; }
}
```